

**(continued)**

Alternatively, you can keep your virtual environments in a central location. The pros of this method are the cons of the other, and vice versa. Your project directory now only contains your project files, but now, you have to remember which project goes with which virtual environment.

There are also alternatives out there to using `venv`: two popular tools for managing virtual environments are `virtualenv` (<https://virtualenv.pypa.io/en/latest/>) and `Conda` (<https://docs.conda.io/en/latest/>). Personally, I use `virtualenv` on my own projects, but it requires an extra install step compared to `venv`. I'm not sure I can even argue in favor of my preference: it was the tool I discovered first, and inertia has kept me there.

### A.3 *Editing tools*

The first line of original code I ever wrote was in the 1980s on an IBM PCjr. It had a whopping 4.77 MHz CPU (that second 7 seemed really important at the time). As I progressed to Unix, my coding world was still very much based on terminals, and to this day, I still prefer to use Vim (<https://www.vim.org/>) while running the Django development server in another terminal.

This isn't for everyone, especially if you're just getting started. The two most popular *integrated development environments* (IDEs) for developing Python are Visual Studio Code and PyCharm. As Python is text, any editor you're happy with will work. If you are using an IDE though, there is some extra configuration that makes your life easier.

IDEs have debugging tools built in. Because Django is a framework, and you're not really running a script, you need to perform extra steps to be able to use the debugger with code run through the Django development server. Once you've created a project in the IDE, you need to modify the run/debug profile to tell it to use the Django development server.

For Visual Studio Code, full instructions on creating a Django project and debugging can be found here: <https://code.visualstudio.com/docs/python/tutorial-django>. The equivalent document for PyCharm is here: <https://www.jetbrains.com/help/pycharm/creating-and-running-your-first-django-project.html>.

If you're using the IDE just to edit the files and running the development server in a terminal, the extra isn't required. If you're used to using the debugger and would miss coding without it, then you'll need to make those changes.

### A.4 *Installing third-party libraries*

Django is more than a framework—it is an ecosystem as well. There are thousands of freely available libraries out there that can make a developer's life significantly better. Don't write something yourself if someone else has already done the work, especially if they've built tests to go with it. The Django Packages site (<https://djangopackages.org/>) is a directory of tools that can augment your Django project, and it has over 4,000 entries. You should check it out to see just what is out there.